

Moving Tech AI Website

Backend documentation movingTechAIBackend / main.py

Version	1.0
Last Updated	04/29/2026

1. Overview

The backend is a Python FastAPI application that sits between the React frontend and Firebase Firestore. It reads software/company profiles from Firestore, normalizes the data, and exposes a REST API for the frontend to consume. The backend is hosted on Render.

2. File Structure

- `main.py` — entire backend: FastAPI app, Firestore client, all endpoints
- `serviceAccountKey.json` — Firebase credentials (never commit to GitHub)
- `requirements.txt` — dependencies: `fastapi`, `uvicorn`, `firebase-admin`

3. How It Connects to Firebase

The backend uses the `firebase-admin` SDK. It supports two modes:

- Local dev: reads `serviceAccountKey.json` from the project folder
- Production (Render): reads credentials from the `GOOGLE_APPLICATION_CREDENTIALS_JSON` environment variable

4. Data Flow

- 1. Frontend calls an API endpoint
- 2. Backend fetches all documents from Firestore `/companies` collection
- 3. Each document is normalized by `_doc_to_dict()` into a consistent shape
- 4. Results are filtered or searched
- 5. Results are sorted across all matched items (before pagination)
- 6. Results are paginated and returned as JSON to the frontend

5. Key Helper Functions

`_doc_to_dict(doc)`

Converts a raw Firestore document into the API shape the React components expect. Renames fields (e.g. `software_typology` -> `typology`), joins arrays into strings, and flattens nested fields.

`_bool_flag(val)`

Converts Firestore boolean `true/false/null` into `'Y' / 'N' / None` strings for the React feature badge components.

`_fetch_all_docs()`

Streams all documents from the `/companies` Firestore collection and converts them using `_doc_to_dict()`.

`_matches_filters(item, filters)`

Checks if a company item passes all active sidebar filters. Text filters use case-insensitive partial matching. Feature filters check for `'Y'` or `'N'`.

`_sort_items(items, sort_by)`

Sorts all matched items BEFORE pagination so ordering is consistent across all pages. Supports: `verified_first`, `unverified_first`, `name_asc`, `name_desc`.

`_paginate(items, page, page_size)`

Slices the sorted list into a page and returns the total count for the frontend pagination component.

6. API Endpoints

GET `/api/software/filter`

Main endpoint used by the directory page. Accepts `typology`, `best_for`, `install`, `language`, `status`, `feature flags` (`lead_mgmt`, `dispatch`, etc.), `sort_by`, `page`, and `pageSize` as query parameters.

GET `/api/software/search`

Full-text search across name, description, and keywords. Also accepts `sort_by` and pagination parameters.

GET `/api/software/options`

Returns all distinct values for filter dropdowns. The frontend calls this on load to populate the sidebar filters dynamically.

GET `/api/software/{company_id}`

Fetches a single software profile by its Firestore document ID slug. Example: GET `/api/software/smartmoving`

7. How the Frontend Connects

The React frontend communicates through `src/api/client.js`. All API calls go through named functions — components never call `fetch()` directly.

- `searchSoftware(q, page, pageSize, sortBy)` → GET `/api/software/search`
- `filterSoftware(filters, page, pageSize, sortBy)` → GET `/api/software/filter`
- `getFilterOptions()` → GET `/api/software/options`
- `getSoftwareById(id)` → GET `/api/software/{id}`

The base URL is set via `REACT_APP_API_URL` environment variable. Defaults to `http://localhost:8000` for local development.

8. Running Locally

1. Place `serviceAccountKey.json` in the `react-app-backend` folder
2. Run: `pip install -r requirements.txt`
3. Run: `python -m uvicorn main:app --reload --port 8000`
4. Visit `http://localhost:8000/docs` for interactive API docs

9. Render Deployment

The backend is deployed on Render. The `GOOGLE_APPLICATION_CREDENTIALS_JSON` environment variable must be set in the Render dashboard with the full JSON content of the Firebase service account key. When new code is pushed to the connected GitHub branch, Render automatically redeploys.